NEXUSFLOW

Complete Project Documentation

Agentic AI-Based Sales Lead Analysis & Follow-Up System Using RAG

A single reference covering the problem, solution, implementation, architecture, technologies, workflows, testing, limitations, and future roadmap.

Project detail	Description
Application	NexusFlow — Sales Intelligence
Type	Full-stack AI / machine-learning prototype
Business use case	Sales lead prioritization and follow-up assistance
Frontend	React + TypeScript + Vite
Backend	Python + FastAPI
Persistence	SQLAlchemy + SQLite; UI reported SQLite WAL connected
AI components	scikit-learn prediction, hybrid RAG, LangGraph workflow
Observed runtime mode	Deterministic Demo Mode
Documentation date	27 October 2026

This guide consolidates project details established in the conversation and observed application behavior. Where code has not been re-inspected for this document, details are described as reported/observed design rather than guaranteed behavior.

Contents

- 1. Executive summary
- 2. Problem statement
- 3. Solution and objectives
- 4. Users and use cases
- 5. Features and pages
- 6. End-to-end workflow
- 7. Architecture
- 8. Tools and technology stack
- 9. Machine-learning component
- 10. RAG and hybrid retrieval
- 11. LangGraph orchestration
- 12. Follow-up generation and approval
- 13. Data and persistence
- 14. Frontend implementation
- 15. Backend and API
- 16. Testing and verification
- 17. Bugs found and fixes
- 18. Limitations and risks
- 19. Running locally
- 20. Demo script
- 21. Resume description
- 22. Interview preparation
- 23. Future roadmap
- 24. Glossary

1. Executive summary

NexusFlow is a full-stack sales-intelligence prototype designed to help sales teams understand leads, prioritize opportunities, retrieve relevant internal knowledge, and prepare follow-up messages. It combines a supervised machine-learning prediction service with a retrieval-augmented knowledge search pipeline and a multi-step agent workflow.

The application presents lead details and engagement indicators, conversion-probability estimates and priority scores, agent execution traces, an indexed knowledge base, follow-up drafts requiring human review, and model/retrieval evaluation views.

The core workflow is: analyze a lead, gather context, retrieve relevant company knowledge, choose a next action, draft a message, validate it, and pause for approval before simulated dispatch.

Plain-English explanation

NexusFlow acts like an AI assistant for a sales team. It estimates which leads may deserve attention, searches internal sales material for relevant context, recommends a next step, and drafts a follow-up email for a human to review.

Current project status

- The local frontend and backend were started successfully.
- The lead-detail bug was fixed; the live UI subsequently displayed 84% probability and a priority score of 91.38 for Alex Morgan, matching the previously inspected analysis value of 0.8388.
- The draft-generation path was changed to remove a hardcoded unsupported outcome claim and improve pain-point text cleanup.
- Reported after the lead-detail fix: 8 lead API tests passed, 22 backend tests passed, and the frontend production build succeeded.
- The app is a prototype. Real-world conversion uplift, probability calibration, and actual email delivery have not been established.

Test results above are reported from the development session. Rerun tests after subsequent changes.

2. Problem statement

Problem

Sales teams may manage many leads with different company profiles, engagement histories, budgets, timelines, and needs. Manually reviewing every lead, deciding whom to prioritize, finding relevant sales guidance, and writing personalized follow-ups can be repetitive and inconsistent.

Pain points addressed

- Lead overload: salespeople need to prioritize limited time.
- Fragmented context: lead attributes, engagement metrics, interaction history, and company knowledge may be stored separately.
- Inconsistent next steps: follow-up choices may depend on individual judgement and may not be documented.
- Repetitive writing: personalized outreach takes time and can contain awkward or unsupported claims.
- Limited transparency: recommendations are difficult to trust if users cannot inspect how they were produced.

Problem statement

How might a sales team use lead data and existing sales knowledge to prioritize opportunities and prepare relevant follow-ups, while keeping decisions inspectable and retaining human control over outreach?

Why AI is relevant

Machine learning can estimate conversion likelihood from lead features; retrieval can surface relevant material from internal documents; workflow orchestration can coordinate these capabilities into a repeatable sequence. The goal is to assist human decision-making, not replace sales judgement.

3. Solution and objectives

NexusFlow connects lead management, predictive intelligence, hybrid knowledge retrieval, next-action selection, draft generation, validation, and human approval in one web application.

Objectives

- Display lead profile, commercial context, engagement telemetry, and interaction history.
- Estimate conversion probability and calculate or retrieve a lead priority score.
- Retrieve relevant knowledge chunks using semantic and keyword search.
- Coordinate multi-step analysis through a LangGraph workflow.
- Generate follow-up drafts that use available lead context and avoid unsupported outcome claims.
- Provide execution traces so users can inspect workflow steps.
- Keep drafts under human review before simulated dispatch.
- Provide evaluation and settings pages for the prototype.

Prototype success criteria

- API and UI show consistent prediction values.
- Agent runs complete and persist their traces.
- Retrieved chunks are relevant and their source can be inspected.
- Drafts are readable and do not invent statistics or customer outcomes.
- Approval/rejection state persists and unapproved drafts cannot dispatch.
- Automated tests cover core behavior and failure cases.

4. Users and use cases

User	Goal	How NexusFlow helps
Sales development representative (SDR)	Decide whom to contact and what to say	Lead context, priority estimate, recommended action, draft email
Account executive	Prepare for an opportunity	Commercial profile, engagement signals, knowledge references
Sales manager	Inspect pipeline and agent behavior	Dashboard, execution history, evaluation view
Sales enablement / operations	Maintain internal knowledge	Indexed knowledge base and retrieval
Reviewer / approver	Control outbound communication	Edit, approve, or reject drafts

Example scenario

A fictional lead, Alex Morgan at TechCorp Global, is shown as a Chief Revenue Officer. The app displays technology-industry context, a $50k–$100k budget tier, a 1–3 month purchase timeline, engagement metrics, and a pain point related to manual data entry and SDR response latency. The inspected analysis contained conversion_probability = 0.8388 and priority_score = 91.38. After the detail-response fix, the UI showed 84%. The agent retrieved five knowledge chunks, selected “Schedule Technical Architecture Demo,” created a draft, ran a contact-policy check, and routed the run to human approval.

This is an observed demo scenario, not evidence that the lead is real or that the model probability is calibrated against actual outcomes.

5. Features and pages

Page	Purpose	Checks to perform
Dashboard	High-level pipeline and follow-up metrics	Reconcile totals with records; verify labels match formulas
Leads & Pipeline	Browse leads, stages, profiles and predictions	Details load; prediction matches latest analysis; sorting/filters work
Agent Activity	Inspect workflow runs and node traces	Inputs, outputs, status and persistence are correct
Follow-Up Center	Review generated follow-up drafts	Edit/save/reject/approve behavior and dispatch safeguards
Knowledge Base	Show indexed documents and chunk counts	Indexing, metadata, source references and retrieval relevance
Model Evaluation	Present model and retrieval metrics	Metrics computed from real evaluation data and explained
Settings	Show application and AI configuration	Displayed settings match active configuration

Observed UI details

- Header displayed “LLM: Deterministic Demo Mode” and “SQLite WAL Connected.”
- Knowledge Base showed documents such as 07 Sales Objections, 13 Case Studies, 11 Email Guidelines, and 10 Sales Playbook.
- A dashboard screenshot showed 711 total leads, 433 high-priority leads, 49.1% conversion rate, 293 overdue follow-ups, and a separate note that 7 drafts awaited review. These figures have not yet been reconciled with underlying records and formulas.
- The dashboard displayed a message saying analytics refresh every 15 seconds; verify actual refetch behavior in code/network activity.

6. End-to-end workflow

Step	Description
1. Load Lead	Load selected lead using a lead tool.
2. Gather Context	Fetch interaction history and related context.
3. Analyze Intelligence	Call prediction and priority-scoring functions.
4. Determine Retrieval Needs	Prepare a search query / identify useful context.
5. Retrieve Knowledge	Search the knowledge base; the inspected run retrieved five chunks.
6. Decide Next Action	Decision engine selected a next action such as scheduling a technical architecture demo.
7. Generate Draft	Create a follow-up draft associated with the lead.
8. Validate Output	Run contact-policy checks; inspected run returned Policy allowed=True and ApprovalRequired=True.
9. Human Approval	Pause for review in Follow-Up Center.
10. Completed	Persist run and final state.

The UI described an “11-node architecture,” while the inspected trace displayed numbered steps through 10, including persistence/completion. If this difference remains in the current code, reconcile the displayed node count with the graph.

Workflow data flow

```
Lead record + engagement context
        ↓
ML prediction + priority score
        ↓
Semantic retrieval + BM25 → RRF-ranked chunks
        ↓
Next-action decision
        ↓
Evidence-aware follow-up draft
        ↓
Contact-policy validation
        ↓
Human approval / rejection
        ↓
Persist run and draft status
```

A completed workflow does not automatically mean that the recommendation is correct or the generated email is well grounded.

7. Architecture

Layer	Components	Responsibility
Presentation	React, TypeScript, Tailwind CSS, Recharts	Pages, forms, charts, traces and draft review
API/application	FastAPI, route handlers, Pydantic schemas	Expose endpoints, validate requests, shape responses
Workflow	LangGraph nodes and tools	Coordinate analysis, retrieval, decision, draft and review
Prediction	scikit-learn model artifact and metadata	Estimate conversion probability
Retrieval	Sentence Transformers, ChromaDB, BM25, RRF	Index and retrieve relevant text chunks
Persistence	SQLAlchemy + SQLite / WAL	Store leads, analyses, interactions, drafts and runs
Quality checks	pytest, API tests, frontend build	Detect regressions and verify expected behavior

Simplified architecture diagram

```
Browser (React + TypeScript)
       | HTTP / API requests
       ▼
FastAPI backend ──────────── SQLAlchemy / SQLite
       |                          └─ leads, analyses,
       |                             interactions, drafts, runs
       ├─ Prediction service → scikit-learn
       ├─ LangGraph workflow → tools / decisions / draft / validation
       └─ Hybrid retrieval → embeddings + ChromaDB + BM25 + RRF
```

This is a logical architecture, not a deployment diagram. Exact module boundaries should be checked against the current repository if code changes.

8. Tools and technology stack

Tool / technology	Category	Role
Python 3.11.9	Language/runtime	Backend and AI services
TypeScript	Language	Typed frontend logic
React 19	UI library	Component-based web interface
Vite	Frontend tooling	Development server and production build
Tailwind CSS	Styling	Utility-based interface styling
React Router	Routing	Navigation between pages
TanStack Query	Server state	API fetching and caching patterns
Axios	HTTP client	Frontend API requests
Recharts	Visualization	Dashboard and evaluation charts
Lucide	Icons	UI icons
FastAPI	Backend framework	REST API
Uvicorn	ASGI server	Runs FastAPI
Pydantic	Validation/schema	Request and response validation
SQLAlchemy	ORM	Database models and queries
SQLite	Database	Local persistence
SQLite WAL	Database mode	Write-ahead logging; UI reported connected
scikit-learn	ML library	Lead conversion prediction
Logistic Regression	ML algorithm	Binary classification / probability output
LangGraph	Workflow framework	Multi-step agent orchestration
Sentence Transformers	Embeddings	Semantic text representations; all-MiniLM-L6-v2 was identified previously
ChromaDB	Vector store	Store/search vector embeddings
BM25	Retrieval method	Keyword-based ranking
Reciprocal Rank Fusion	Ranking method	Combine semantic and keyword rankings
pytest	Testing	Backend automated tests
FastAPI TestClient	API testing	Exercise API routes in tests
Git	Version control	Track source changes
VS Code	IDE	Development and debugging
GitHub Copilot Chat	AI coding assistant	Code generation, inspection, edits and test assistance
ChatGPT	AI assistant	Planning, explanations, debugging guidance and documentation

This table describes reported roles, not a guarantee that each technology is imported by every module. Package manifests are the source of truth for exact installed versions.

AI-assisted development

The project was developed with substantial AI assistance using ChatGPT and GitHub Copilot Chat. AI tools helped generate and revise code, inspect failures, propose fixes, and assist with tests. Be clear about which decisions and technical details you personally understand, configured, tested, and can explain.

9. Machine-learning component

Purpose

The prediction service estimates a lead's likelihood of conversion. A priority score is also shown to help rank leads for attention.

Known implementation elements

- The model artifact is loaded from artifacts/lead_model.joblib.
- Preprocessing/model metadata is loaded from artifacts/preprocessor_meta.json.
- Logistic Regression was identified as the model in prior code investigation.
- The prediction service uses model probability output (predict_proba was identified previously).
- Alex Morgan's inspected analysis contained probability 0.8388 and priority score 91.38; the corrected UI displayed 84%.

Conceptual flow

1. Collect lead features supported by the trained model.
2. Apply the preprocessing expected by the model.
3. Generate the conversion probability.
4. Calculate or retrieve the priority score.
5. Persist the analysis and expose it through the API and UI.

Interpretation and evaluation

An 84% display means the model returned a value around 0.84 for that input. It does not guarantee that the lead will convert. Reliable probability interpretation requires evaluation on representative held-out data and calibration analysis.

- Document training data provenance and representativeness.
- Check train/test separation and data leakage.
- Measure accuracy, precision, recall, F1, ROC-AUC and class balance.
- Assess calibration and compare with a simple baseline.
- Review performance across lead segments and over time.

Earlier code inspection referenced approximate evaluation figures of 56.7% accuracy and 0.609 ROC-AUC. Do not present these as definitive without rerunning evaluation and verifying dataset/methodology.

10. RAG and hybrid retrieval

What RAG means here

Retrieval-Augmented Generation retrieves relevant text from a knowledge base and supplies it as context for generation. It can make outputs more relevant to internal material, but retrieval alone does not guarantee factual correctness.

Hybrid retrieval pipeline

6. Documents are ingested and split into text chunks.
7. Sentence Transformers creates embeddings for semantic retrieval.
8. ChromaDB stores and searches vector representations.
9. BM25 performs keyword-based retrieval.
10. Reciprocal Rank Fusion combines the result rankings.
11. The agent receives retrieved chunks and metadata as context for draft generation.

Knowledge Base examples observed in the UI

Document label	Category shown	Chunks shown
14 Product Limitations	Technical	4
13 Case Studies	Case Studies	7
12 Discount Policy	Policy	3

11 Email Guidelines	Guidelines	5
10 Sales Playbook	Playbook	5
09 Onboarding Guide	Customer Success	4
08 Competitor Comparison	Competitive	5
07 Sales Objections	Objections	12
06 Security Compliance	Security	3

These labels and counts were visible in a screenshot; they may not represent the complete knowledge base.

RAG quality checks

- Inspect retrieved passage text, not just document names.
- Verify that each generated claim is supported by the actual passage.
- Measure Recall@k and MRR using a defensible, human-reviewed relevance set.
- Test empty, irrelevant, ambiguous and conflicting retrieval results.
- Do not treat source names or metadata as evidence.

11. LangGraph agent orchestration

LangGraph coordinates a stateful workflow in which nodes perform tasks and pass state between steps. NexusFlow's UI shows node-level trace information, tool names, inputs, outputs, durations and run status.

Why use an agent workflow?

- Separates work into inspectable steps rather than one opaque function.
- Allows routing to human approval after draft validation.
- Provides trace data for debugging.
- Allows individual steps to be tested and improved.

Demo mode distinction

The running app explicitly displayed “LLM: Deterministic Demo Mode.” At least some action selection and drafting behavior is rule/template-based in this mode. A configured LLM path may also exist, but describe the actual mode used for a particular run accurately.

Workflow completion, output correctness, grounding and business usefulness are separate evaluation dimensions.

12. Follow-up generation and human approval

Intended flow

12. Select a lead and run the workflow.
13. Use lead context and retrieved knowledge to create a follow-up draft.
14. Validate contact policy and determine whether approval is required.
15. Present the draft for review in Follow-Up Center.
16. Allow a human to edit, approve, or reject the draft.
17. Record the status and any simulated dispatch event.

Grounding bug found and corrected

The original email contained a hardcoded sentence claiming that companies saw “immediate operational velocity gains.” Investigation found it was a fixed prompt/template string rather than a claim supported by the cited document. The email also copied the raw pain-point text (“manual data entry bottlenecks sdr response latency”) awkwardly and exposed an internal source label (“07 Sales Objections”).

The generation path was changed to remove the hardcoded outcome claim, pass retrieved text with metadata, and clean the raw pain-point wording. A fresh draft then read:

```
Hi Alex,

You mentioned: Manual data entry is bottlenecking SDR response latency.

A possible next step is to schedule Technical Architecture Demo.

Would you be open to discussing this further?

Best regards,
NexusFlow Sales Intelligence Team
```

This confirmed removal of the unsupported claim and improved the pain-point text, but the draft remained generic and awkward. Email tone and natural calls to action remain an improvement opportunity.

Safety and limitations

- A generated draft is not automatically safe or ready to send.
- The inspected run returned Policy allowed=True and ApprovalRequired=True and then routed to human approval.
- Earlier code inspection raised concerns that some lower-level simulated-send paths or status transitions may not enforce approval consistently. Test the dispatch boundary, not just the UI.
- Dispatch is simulated; do not claim real email delivery without a verified email integration.
- Claims about outcomes, customers, metrics, company priorities, dates or availability should only be included when supported by reliable context.

13. Data and persistence

The backend uses SQLAlchemy with SQLite. The UI reported SQLite WAL Connected. The project persists leads, analyses, interactions, drafts and agent runs/traces; exact table names should be confirmed in backend/app/models/orm_models.py.

Why persistence matters

- Lead data remains available after refresh.
- Agent traces remain inspectable after completion.
- Draft content and review status persist.
- Analysis values are consistent across pages.

Lead-detail probability bug

The lead-detail endpoint initially returned the lead ORM object without enriching it with the latest persisted analysis, while the list route had separate enrichment behavior. The UI defaulted a missing conversion_probability to zero, so the page showed 0% even though the stored analysis and agent trace showed 0.8388 and priority score 91.38.

The fix updated detail-response enrichment to expose the latest analysis, use the existing fallback behavior when no analysis exists, and represent an unavailable prediction as null so the UI can display N/A instead of a false 0%. The live page subsequently displayed 84% and 91.38 for Alex Morgan.

Dashboard metric caution

A dashboard screenshot showed 711 total leads, 433 high-priority leads, 49.1% conversion rate and 293 overdue follow-ups, with 7 drafts awaiting review. These figures still require reconciliation. “Closed-won conversion” must reflect actual outcomes, not simply predicted probabilities.

14. Frontend implementation

The frontend is a React/TypeScript single-page application built with Vite and styled with Tailwind CSS. The visual design uses a dark indigo sidebar, light workspace background, white cards, and purple/teal accents.

- Route navigation between dashboard, leads, agent activity, follow-up center, knowledge base, evaluation and settings.
- Fetch and display backend data.

- Render charts, summary metrics, lead profiles, predictions and agent traces.
- Support draft editing and review controls.
- Display knowledge documents and evaluation metrics.

The frontend production build succeeded after the recent fixes. A pre-existing large-bundle warning was reported. A successful build verifies compilation/bundling, not every live interaction.

15. Backend and API

The backend uses FastAPI and Uvicorn. Route handlers expose data/actions; schemas validate and shape API payloads; services encapsulate prediction, scoring, retrieval and draft logic; LangGraph coordinates the workflow.

Path / module	Role
backend/app/main.py	Application startup, configuration and route registration
backend/app/api/routes/leads.py	Lead API routes, including detail retrieval
backend/app/services/lead_service.py	Lead retrieval and analysis enrichment
backend/app/schemas/lead_schemas.py	Lead response schemas
backend/app/services/prediction_service.py	Loads model and computes probability
backend/app/services/scoring_service.py	Lead priority scoring
backend/app/api/routes/analyses.py	Analysis-related routes
backend/app/models/orm_models.py	SQLAlchemy persistence models
backend/app/agents/prompts.py	Prompt/template content; included the previous hardcoded claim
backend/app/agents/nodes.py	Agent workflow node logic, based on prior inspection
backend/tests/conftest.py	Shared pytest fixtures
backend/tests/test_api.py	API tests including lead-detail regression coverage

Local endpoints observed

- Frontend: http://localhost:5173/
- Backend: http://127.0.0.1:8000
- Interactive API docs: http://127.0.0.1:8000/docs

Inspect /docs for the current endpoint list. This guide avoids inventing route paths beyond the modules confirmed during investigation.

16. Testing and verification

Test run	Reported result	Scope
Earlier baseline	13 tests passed in a previous session	Agent, API, prediction, retrieval and scoring
After grounding change	19 tests passed (reported earlier)	Grounding and agent behavior
After lead-detail fix	8 lead API tests passed	Latest analysis, fallback, unavailable prediction
After lead-detail fix	22 backend tests passed	Full backend suite
Frontend build	Passed	Production build; existing large-chunk warning

The latest reported result is 22 backend tests passing and a successful frontend build. Tests used a temporary SQLite database; the project database and Chroma index were reported as untouched.

Manual test checklist

- Dashboard: reconcile totals, high-priority count, conversion rate and overdue follow-ups with backend calculations.
- Leads: verify probability and priority match the latest analysis; create a fictional test lead and check persistence.
- Agent Activity: run an agent and inspect each node; verify the trace persists after refresh.
- Knowledge Base: search a topic and inspect actual retrieved passage content.
- Follow-Up Center: edit/save/reject a draft; verify status persistence and dispatch restrictions.
- Model Evaluation: confirm metrics are calculated from a defined evaluation dataset.
- Settings: verify displayed values match active configuration.
- Browser console/network: inspect recurring errors and failed API requests.

Test command

```
cd backend
python -m pytest -q
```

Run from the project root's backend directory if imports expect app to resolve there. Confirm the test configuration before changing environment variables. Avoid deleting or recreating the development database or Chroma index as a troubleshooting shortcut.

17. Bugs found and fixes

Issue	Cause / observation	Fix or status
Lead page showed 0% while trace showed 83.88%	Detail API omitted latest analysis; UI defaulted missing value to zero.	Fixed. Detail API exposes latest analysis; unavailable probability becomes null/N/A. Live UI showed 84% and 91.38.
Email made unsupported outcome claim	Hardcoded template said companies saw “immediate operational velocity gains.”	Removed from generation path; grounding regression tests added.
Raw pain point appeared awkwardly	Raw CRM text was inserted without natural-language cleanup.	Basic cleanup added; newer draft clearer but still generic.
Internal source label leaked into email	Draft exposed document label as customer-facing text.	Generation approach changed; verify future drafts.
Dashboard metrics not reconciled	Screenshot cannot establish formulas or correctness.	Open investigation: trace each metric to API, database fields and formula.
ChromaDB telemetry warning	Startup logged a telemetry capture argument error.	App still started; root cause not established here.

Engineering lesson

A generated application must be tested beyond the happy path. UI defaults can hide missing data, document names can be mistaken for evidence, and a completed workflow can still produce a poor recommendation.

18. Limitations and risks

- Predictive performance and calibration have not been independently established for a real deployment.
- The observed environment may use demo/synthetic data; validate dataset provenance before claiming real customer impact.
- RAG quality depends on document quality, chunking, ranking, and evidence support.
- Prior inspection noted that evaluator relevance logic may accept keyword matches as relevant; this can overstate retrieval quality without careful labels.
- Feature-contribution indicators were previously identified as heuristic, not formal model explainability.
- A confidence indicator based on distance from 0.5 is not necessarily calibrated confidence.
- Deterministic demo mode uses rules/templates for at least some behavior.
- Simulated dispatch is not actual email delivery.
- Approval safety must be enforced at the dispatch/action boundary.
- Dashboard formulas and counts still need reconciliation.
- The frontend build has had an existing large-bundle warning.
- The app is a prototype, not production-ready until security, privacy, reliability, monitoring, evaluation and integrations are addressed.

19. Running locally

Prerequisites

- Python 3.11.x
- Node.js and npm compatible with frontend/package.json
- Dependencies installed from the project's requirements/package files
- Project opened in VS Code or another terminal

Start the backend

Open PowerShell terminal 1 and run:

```
cd "C:\GEETHU\COLLEGE\7th SEM\BLUVERSE AGENTIC AI\sales_project\backend"
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Wait for the application startup completion message. Running app.main:app from the project root previously produced ModuleNotFoundError because the app package is inside backend.

Start the frontend

Open a second terminal and run:

```
cd "C:\GEETHU\COLLEGE\7th SEM\BLUVERSE AGENTIC AI\sales_project\frontend"
npm run dev
```

Open

- Frontend: http://localhost:5173/
- API docs: http://127.0.0.1:8000/docs

Keep both terminals open during local use. If startup fails, inspect the current directory and first traceback before reinstalling packages or recreating the environment.

20. Demo script

18. Start on the Dashboard and explain the pipeline metrics, noting any metric still under validation.
19. Open a lead and explain its commercial context and engagement signals.
20. Show conversion probability and priority score; explain that these are model outputs, not guarantees.
21. Open Agent Activity, run the agent and explain context, prediction, retrieval, action selection, draft, validation, approval and persistence.
22. Open Knowledge Base and explain semantic plus keyword retrieval.
23. Open a generated follow-up and explain the grounding bug that was discovered and fixed.
24. Show the pending-approval state and explain that dispatch is simulated.
25. Mention regression tests and the lead-detail API/UI consistency fix.

21. Resume-ready project description

Suggested title

NexusFlow — Agentic AI Sales Intelligence & Lead Follow-Up System

Suggested bullets

- Developed a full-stack sales-intelligence prototype combining ML-based lead conversion prediction, lead prioritization, and follow-up draft generation.
- Implemented hybrid RAG using Sentence Transformers, ChromaDB, BM25, and Reciprocal Rank Fusion to retrieve relevant sales knowledge.
- Orchestrated a multi-step LangGraph workflow for lead analysis, retrieval, next-action selection, draft generation, validation, and human review.
- Built a React/TypeScript interface and FastAPI backend for lead management, prediction insights, agent traces, knowledge-base inspection and draft review.
- Added regression tests for evidence-grounded generation and lead-analysis consistency; the latest reported backend suite had 22 passing tests.

Honest positioning

AI tools contributed substantially to implementation. Be clear about which decisions and technical details you personally understand. Do not claim real conversion uplift, production email sending, or model performance without evidence. Only list technologies you can explain.

Compact technical skills line

Python, TypeScript, React, FastAPI, SQLAlchemy, SQLite, scikit-learn, LangGraph, RAG, Sentence Transformers, ChromaDB, BM25, Reciprocal Rank Fusion, pytest, Git, GitHub Copilot, VS Code.

22. Interview preparation

30-second explanation

“NexusFlow is an AI-assisted sales-intelligence application. It uses a machine-learning model to estimate lead conversion probability, retrieves internal sales knowledge through hybrid RAG, and orchestrates a workflow that recommends a next action and prepares a follow-up draft. A human reviews the draft before simulated dispatch. I also added tests and investigated issues such as unsupported claims and inconsistent prediction values between the API and UI.”

Likely questions

Question	Answer outline
Why combine ML and RAG?	ML estimates probability from structured features; RAG retrieves unstructured internal material to contextualize recommendations and drafts.
Why semantic search and BM25?	Semantic search finds conceptually related text; BM25 is strong for keywords. RRF combines their rankings.
What does LangGraph contribute?	Stateful multi-step orchestration, inspectable transitions and an approval point.
What bug did you find?	The detail API omitted the latest analysis and the UI defaulted missing probability to zero. The API was enriched and unavailable values now display N/A.
How did you improve reliability?	Removed a hardcoded unsupported outcome claim, improved pain-point cleanup and added regression tests.
Does 84% mean the lead will convert?	No. It is a model output; calibration and representative held-out evaluation are needed.
Does the app send real email?	The observed dispatch behavior is simulated; real delivery has not been established.
What would you improve next?	Validate dashboard formulas, evaluate the model, label retrieval relevance, strengthen grounding, enforce approval at dispatch, and improve email quality.

23. Future roadmap

Priority	Improvement	Evidence of completion
P0	Reconcile dashboard metrics and labels	Metric-by-metric API/formula audit and tests
P0	Enforce approval/contact policy at every dispatch path	Tests prove unapproved/restricted drafts cannot dispatch
P1	Evaluate model on held-out data and against a baseline	Documented split, baseline, metrics and calibration
P1	Improve claim-level grounding	Evidence-supported claims and tests for unsupported metrics
P1	Create labelled retrieval relevance set	Recall@k/MRR measured against reviewed relevance labels
P1	Improve email tone and call to action	Quality rubric and tests for awkward/internal wording
P2	Add observability and clearer errors	Structured logs, error states and latency monitoring
P2	Optimize frontend bundle	Bundle analysis and justified lazy-loading
P2	Add real email integration only with consent and safeguards	Test environment, audit logs, rate limits and explicit approval

24. Glossary

Term	Plain-English definition
API	Interface that lets frontend and backend exchange data.
BM25	Keyword-based document ranking method.
ChromaDB	Vector database for storing/searching embedding vectors.

Conversion probability	Model estimate that a lead converts; not a guarantee.
Embedding	Numeric vector representation of text used for semantic similarity.
FastAPI	Python framework for building APIs.
Human-in-the-loop	A person reviews or approves an AI-produced action.
LangGraph	Framework for stateful, multi-step agent workflows.
Logistic Regression	Supervised-learning algorithm used for classification/probability estimates.
MRR	Mean Reciprocal Rank; rewards relevant results near the top.
RAG	Retrieve source material and provide it as context for generation.
Recall@k	Fraction of relevant items retrieved in the top k results.
RRF	Method for combining ranked results from multiple retrieval systems.
SQLite WAL	SQLite write-ahead logging mode.
SQLAlchemy	Python SQL toolkit and ORM.
Sentence Transformers	Library for producing text embeddings.
Uvicorn	ASGI server commonly used to run FastAPI.
pytest	Python testing framework.

Final project status summary

NexusFlow is a functioning local prototype with a React frontend, FastAPI backend, predictive lead analysis, hybrid knowledge retrieval, LangGraph orchestration, follow-up drafts, human-review UI, and automated backend tests. Recent fixes addressed a missing lead-analysis enrichment bug and a hardcoded unsupported email claim.

The most valuable next step is not simply adding more features. It is validating correctness and usefulness: reconcile dashboard metrics, verify approval enforcement, measure model performance, assess retrieval relevance, and ensure generated claims are supported. This is how a feature-rich demonstration becomes a defensible engineering project.

Treat this as a living reference. Update package versions, routes, metrics, test counts and implementation notes as the code changes.